The Islamia University of Bahawalpur
Department of Information Technology

INFT-4041 · BSIT · 5th Semester · Fall 2026

E-Commerce Application Development

Chapter

10

The Checkout and the Order State Machine

Part III — From Basket to Order

Taught in	Week 8
Laboratory	Midterm, then queries counted per checkout step
Assessment	**Midterm (Ch 1–9)**

This is a class extract. It is *Chapter 10* of the unified handout, printed on its own so that a week's reading is a week's reading. Page numbers restart at 1 and are local to this extract; chapter, figure and table numbers are the ones the full volume gives them, so a reference you follow here lands in the same place there.

Cross-references to other chapters print as their numbers in the full volume. For the whole course in one file — front matter, the HEC tracing table, all twenty-three chapters and all seven appendices — see `ECAD_Fall2026_Handout.pdf`.

Dr. Mustafa Hameed · Department of Information Technology

Generated from `parts/` by `makechapters.py`; do not edit this PDF's source by hand

Chapter 10

The Checkout and the Order State Machine

Part 1 | Part 2 | **Part 3** | Part 4 | Part 5 | Part 6

Part III — From Basket to Order

Learning Outcomes

By the end of this chapter you will be able to:

- **Name** the steps of a checkout in order, and **state** what each one validates and what it saves.
- **Explain** why an order is a state machine rather than a record, and **read** a shop's order statuses as the states of one.
- **Measure** the cost of each checkout step in time, queries and writes, and **identify** where an order is actually created.
- **Diagnose** a checkout that fails silently, using the shop's own responses rather than its screens.
- **Judge** where a checkout can safely be shortened, and where shortening it would lose something the shop needs.

Before You Start

Chapter 9: where the basket lives, and that it is rows keyed by a session identifier. Chapter 6: reading the queries a page actually ran. Chapter 5's container, with a shop you can place a test order in.

Key Terms in This Chapter

checkout • checkout step • guest checkout • order • order status • state machine • order history • confirmation • idempotency • abandonment • conversion funnel • write amplification • silent failure

10.1 A Checkout Is a Sequence, Not a Page

The basket in Chapter 9 is a list of rows. An **order** is something else entirely: a commitment, by a named person, to pay a stated amount for stated goods delivered to a stated address. Turning the first into the second is what a checkout does, and it cannot be done in one step because each piece of information constrains the next — the delivery address decides which shipping methods exist, the shipping method decides the total, and the total decides what the customer is agreeing to.

Order, Order Status, Order History

An **order** is a row recording an intention to buy, together with rows for the products, the totals and the addresses as they were *at the moment of ordering*. It does not point at the catalogue; it copies from it, because the price of a product next month must not change what a customer agreed to pay today.

An **order status** is the state the order is in: pending, processing, shipped, complete, refunded. The set of statuses is the shop's, not the standard's.

The **order history** is the list of status changes, each with a date and an optional comment. The status column says where the order is; the history says how it got there, and it is the only part a dispute can be settled from.

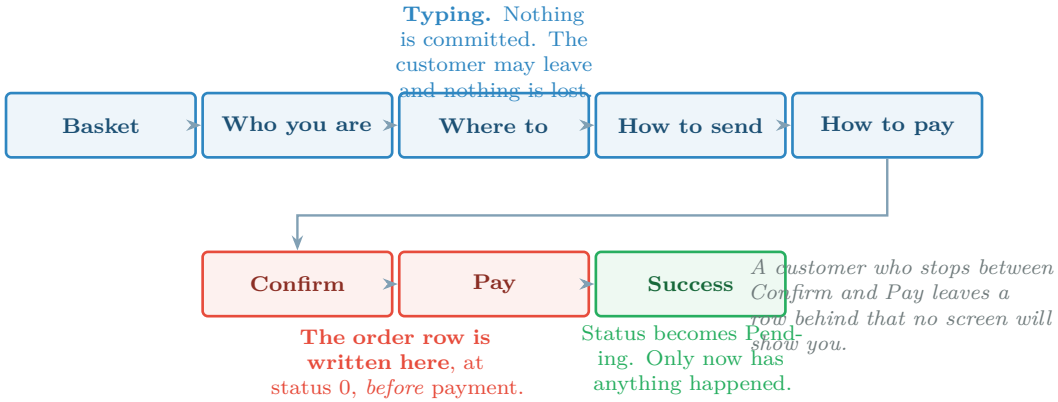


Figure 10.1: The guest checkout as the shop actually runs it. The first five steps collect; the last three commit. The gap between the order row appearing and the payment confirming it is where orders go missing.

10.2 The Fault That Stops Every Order

Before any of that could be measured, the checkout had to be made to work, and on a freshly installed shop it does not. This is worth the space because the way the failure presents itself is the lesson, not the fault itself.

A stock install cannot complete a guest checkout

Driving the sequence by hand against the shop built in Chapter 5, every step succeeded until the shipping quote, which answered:

```
{"error":"Shipping address required!"}
```

— although a shipping address had been supplied and accepted one step earlier, with `{"success":"... has been saved!"}`.

The cause. The guest registration step copies the billing address to the shipping address, but only inside a branch guarded by the setting `config_checkout_payment_address`. That setting is read by **ten** controllers in `catalog/`, and appears as a checkbox in the admin settings form — but **the installer never writes it**. An absent setting reads as false, the branch never runs, no shipping address is ever stored, and the quote has nothing to work from.

The fix is to open **System › Settings › Edit › Option**, tick **Payment Address required** and save, which writes every field on that form, including the missing one. One checkbox.

What makes this worth a page. No page returned an error code. No log recorded a failure. Every screen rendered. A shopkeeper would see customers reaching the shipping step and leaving, and would reasonably conclude that the shipping charges were too high. This is the same shape of fault as the store identifier in Chapter 6: **the system did not break, it answered a different question**, and only driving it end to end and reading the replies found it.

How to find a fault of this shape

1. **Read the reply, not the page.** Each step answers with JSON. The error was in the reply all along; the browser merely rendered it in a place nobody looks.
2. **Trust the sequence, not the screens.** Drive the steps in order from a script. The step that first disagrees with the previous one is the step to read.
3. **Then read the source for that step only.** Not before — the measurement tells you which of four thousand files to open.
4. **Change one thing and repeat.** Setting the one flag made the whole sequence complete, which is what turns a hypothesis into a cause.

10.3 What Each Step Costs

With the checkout working, the sequence can be measured. The question put to it is a specific one: *is the last step the expensive one?* It is the common belief, and it is worth settling with numbers.

Measured: A complete guest checkout, step by step

Produced by `code/m10_checkout.py -repeat 5` against the stack in Appendix A: medians of five complete checkouts, each buying two products. Queries and writes are counted from MariaDB's slow query log, not estimated.

Step	Time	Queries	Writes	DB time	Share
fill basket	55 ms	57	6	23 ms	11%
register (guest)	30 ms	41	3	9 ms	6%
shipping address*	21 ms	32	2	6 ms	4%
shipping method	56 ms	67	4	22 ms	11%
payment method	54 ms	74	4	17 ms	10%
confirm	**149 ms**	**125**	**25**	93 ms	**28%**
pay	64 ms	102	8	25 ms	12%
success	94 ms	109	3	37 ms	18%
Total	**523 ms**	**607**	**55**	—	

* *A guest never uses this step.* It redirects to the login page, because its guard requires a logged-in customer. The round trip costs 21 ms and 32 queries and achieves nothing; a guest's addresses were both set by the registration step.

The common belief is wrong as stated. No single step costs more than the rest together. `confirm` is the most expensive at 28%, not 50%.

But split the sequence where the customer stops typing:

	Time	Queries	Writes
Typing (5 steps)	217 ms	271	19
Ordering (3 steps)	307 ms	336	**36**

The three steps after the last keystroke cost more than the five before it — 40% more time, and nearly twice the writes. That is the defensible version of the claim.

The sharpest number in the table is the writes column. `confirm` performs **25 of the 55 writes** in the entire checkout, against two to six for every other step. It is not slow because it is complicated; it is slow because it is the step that *writes the order*: the order row, the product rows, the totals, the addresses, all copied out of the basket and the session so that they can never change again.

For scale: one checkout is **607 queries**. A category page in Chapter 6 was 144. A single customer buying two items costs the database more than four people browsing.

10.4 An Order Is a State Machine

The reason `confirm` and `pay` are separate steps — and the reason the gap between them matters — is that an order has a life after it is created, and that life is a sequence of states.

</> The fourteen states this shop defines

System › Localisation › Order Statuses lists them. They are not a standard; they are this shop's vocabulary, and a shop may add its own.

1 Pending	8 Denied	13 Chargeback
2 Processing	9 Canceled Reversal	14 Expired
3 Shipped	10 Failed	15 Processed
5 Complete	11 Refunded	16 Voided
7 Canceled	12 Reversed	

Three things are worth noticing. **There is no 4 and no 6** — the numbers are identifiers, not an order. **Several are about money moving backwards** (Refunded, Reversed, Chargeback, Canceled Reversal), which is Chapter 18's subject and a reminder that selling is the easy half. **And there is no zero.**

Measured: Status 0, and the orders nobody can see

From `code/m10_checkout.py -abandon`, which runs the checkout as far as `confirm` and then stops, exactly as a customer who changes their mind at the payment button does.

Orders at status 0 before	0
Orders at status 0 after	**1**
Order row written?	**yes**, total $854.40
Visible in the admin order list?	**no**

The order row is written at `confirm`, before any payment, and left at status 0. Status 0 is not one of the fourteen. The admin order list filters it out. Nothing will ever remove it.

This is not a bug; it is a deliberate design, and the reason for it is sound. The order number has to exist before the customer is sent to a payment provider, because the provider must be told what it is being paid for, and the answer that comes back has to find its way to something. The cost of that design is a table containing rows for orders that never happened.

Why it matters to you: a shop owner reporting "we have orders in the database that aren't in the orders list" has not found a fault. They have found their abandonment rate. Chapter 17 turns that into a number worth watching, and the ratio of status-0 rows to real orders is one of the most useful figures a small shop can compute from data it already has.

Two requests, one order — the problem confirm cannot solve alone

If the customer's connection drops after `confirm` is sent but before the reply arrives, their browser may retry. If the shop treats the second request as a new one, the customer is charged twice.

The property that prevents this is **idempotency**: performing an operation twice has the same effect as performing it once. OpenCart approaches it by keeping the order identifier in the session and updating that order rather than creating another, which is why `confirm` can be called repeatedly in a session without multiplying orders — and why the order identifier must be established *before* payment, not after.

It is not a complete answer, and Chapter 14 returns to it where it is hardest: when the second request comes not from the customer but from the payment provider, minutes later, over a connection that is not the customer's at all.

10.5 Where a Checkout Can Be Shortened, and Where It Cannot

Every step is a place to lose a customer, so the pressure to remove steps is constant. The measurements say which removals are free.

Four changes, ranked by what the evidence supports

1. **Remove the guest's shipping-address round trip.** It is measured at 21 ms and 32 queries, it redirects to a login page the guest cannot use, and the data it would set is already set. Pure waste, and the only change here with no trade-off.
2. **Merge the shipping and payment method steps.** 110 ms and 141 queries between them, and both are selections from short lists. The cost is that a shipping choice can change the total that the payment step displays, so they must then be recomputed together.
3. **Skip the address entirely for digital goods.** There is nothing to deliver. This is a real saving and the platform supports it — `hasShipping()` is already consulted.
4. **Remove the confirmation screen.** Tempting — 28% of the time. **Do not.** It is the last point at which the customer sees what they are agreeing to pay, and removing it moves every mistake from before the sale to after it, where it costs a refund, a complaint and Chapter 22's attention rather than a click.

Common Pitfalls

- **Testing a checkout only through the browser.** The fault at the start of this chapter was invisible there and obvious in the replies.
- **Assuming the slow step is the last step.** It is the most expensive single step, at 28%, which is not the same claim.
- **Counting orders without filtering status 0,** and reporting revenue that includes abandoned baskets.

- **Deleting status-0 rows to tidy up.** They are the audit trail of every customer who reached the payment button, and a payment provider's late reply may still be looking for one of them.
- **Pointing an order's product rows at the catalogue** instead of copying names and prices into the order. The catalogue changes; what a customer agreed to must not.
- **Removing the confirmation screen to raise conversion.** It raises conversion and refunds together.

Try It Yourself: Drive a checkout, then break it on purpose

```
# --- 1. Make the shop able to take an order at all -------------------
# Admin: System > Settings > Edit > Option. Tick "Payment Address
# required" and save. Write down what you expect that to change.

# --- 2. Place one order by hand, in the browser ----------------------
# Note how many pages you visited and roughly how long it took.

# --- 3. Now measure it -----------------------------------------------
cd code
python m10_checkout.py --repeat 5

# --- 4. Predict before you look --------------------------------------
# Which step has the most WRITES? Most people say "success".

# --- 5. Watch an order appear before it is paid for ------------------
python m10_checkout.py --abandon

# --- 6. Look for it in the admin panel -------------------------------
# Admin: Sales > Orders. It is not there. Now look in the table:
docker compose exec db mariadb -ushop -pshoppw opencart \
  -e "SELECT order_id, order_status_id, total FROM oc_order;"

# --- 7. Read the state machine off your own data ---------------------
python m10_checkout.py --states

# --- 8. Move an order by hand through three states -------------------
# Admin: Sales > Orders > View > History. Pending -> Processing ->
# Shipped. Then read oc_order_history and say what the status column
# alone would not have told you.

# --- 9. Clean up -----------------------------------------------------
python m10_checkout.py --clean
```

What to notice. Step 1 is the whole of Chapter 4's argument in one checkbox: a platform you did not write will have defects, and the skill that matters is finding them from the outside, not avoiding platforms.

Checkpoint — Test Yourself

1. A shop reports that customers abandon at the shipping step. Describe, in order, the three things you would do before accepting that the shipping charges are too high — and say what the first chapter measurement would have told you.
2. `confirm` is 28% of the checkout's time but 45% of its writes. Explain why those two shares differ, and what that implies for which step you would optimise if the shop became ten times busier.
3. Your client asks you to delete every order at status 0 because they "clutter the database". Give two distinct reasons to refuse, and one thing you would build instead.

Chapter Summary

- A checkout is a sequence because each answer constrains the next. An order copies what it needs out of the catalogue rather than pointing at it, so that later price changes cannot rewrite an agreement.
- Measured: a stock install *cannot complete a guest checkout*. The setting `config_checkout_payment_address` is read by ten controllers and written by none, so no shipping address is ever stored. Every page still returned 200.
- The way to find a fault of that shape is to drive the steps from a script and read the replies, then open the source for the one step that disagreed.
- Measured: 523 ms and 607 queries for one complete checkout — more than four category pages.
- Measured: no single step costs more than the rest together, but the three steps after the last keystroke cost 307 ms against 217 ms and nearly twice the writes.
- Measured: `confirm` performs 25 of the checkout's 55 writes, because it is the step that creates the order.
- Measured: abandoning at the payment button leaves an order row at status 0 — a status the shop does not define, that the admin list hides, and that nothing removes. It is the abandonment rate, not a fault.
- The order history, not the status column, is what a dispute is settled from.
- A step may be removed when the evidence says it does nothing. The confirmation screen is not that step.

Review Questions

1. **(MCQ)** In the measured checkout, which step performs the most database writes? (a) fill basket; (b) payment method; (c) confirm; (d) success.

2. **(MCQ)** An order sitting at status 0 means: (a) the order failed; (b) the order was refunded; (c) the order row was created but payment was never confirmed; (d) the status table is corrupt.

3. **(Short)** Explain why an order copies the product name and price instead of referring to the catalogue row, and give one thing that would go wrong if it did not.

4. **(Short)** Define idempotency, and explain why a checkout needs it in terms of a specific thing that would otherwise happen to a customer.

5. **(Short)** The stock shop answered every request with 200 while being unable to take a single order. Explain what this says about monitoring a shop by availability.

6. **(Applied)** You are given a shop with 1,200 rows in `oc_order`, of which 820 are at status 0. Say what you can and cannot conclude from that ratio, what two further figures you would want, and what you would recommend — with the reasoning for each step shown.

7. **(Applied)** Design a measurement that would tell you at which checkout step customers actually leave. State what you would record, where in the sequence you would record it, how you would avoid counting the same visitor twice, and how long you would need to run it before the answer meant anything for a shop taking thirty orders a day.